

Programmeren: algoritmen en structuren

31 oefeningen op papier, met een antwoordblad achteraan.

Naam _____ Datum _____

- Schrijf met potlood, dan kan je gerust iets uitgommen en opnieuw proberen.
- Schrijf je tussenstappen op. Een antwoord zonder berekening is niet na te kijken, ook niet door jezelf.
- Zet bij elk antwoord de eenheid, als er een is. Een getal zonder eenheid is in een vraagstuk zelden een antwoord.
- Het antwoordblad zit achteraan. Scheur het eraf voor je begint.

1 De bouwstenen

Schrijf bij elke omschrijving de naam van de bouwsteen.

1. Welke bouwsteen?

a) een naam waarachter een waarde zit die kan veranderen

b) een waarde die tijdens het programma niet verandert

c) code die alleen loopt als iets waar is _____

d) hetzelfde stuk code herhalen _____

2. Hoe heet een stuk code dat je een naam geeft en hergebruikt?

2 Het juiste sleutelwoord

Schrijf het sleutelwoord op dat je in Python gebruikt.

3. Welk sleutelwoord?

- a) een stuk code alleen laten lopen als iets waar is _____
 - b) over alle elementen van een rij lopen _____
 - c) een functie schrijven _____
 - d) teruggeven wat een functie uitrekent _____
-

3 De juiste datastructuur

Kies de datastructuur die het best past.

4.	Wat je wil bewaren	Datastructuur
	de namen van de leerlingen, op volgorde, nog aan te passen	_____
	bij elke leerling zijn punt	_____
	alle verschillende voornamen, zonder dubbels	_____
	de coördinaten van een punt, die vastliggen	_____

4 Wat blijft er over in een set?

Je steekt deze lijst in een set. Hoeveel elementen telt die set?

5. Hoeveel?

- a) [2, 5, 5, 7, 7, 7] _____ b) [1, 2, 3, 4] _____
- c) [9, 9, 9, 9] _____ d) [0, 1, 0, 1, 2] _____

5 Waar of niet waar: datastructuren

Kruis aan.

6. Een **tuple** kan je achteraf nog wijzigen.
- ☐ waar ☐ niet waar
7. Een **list** bewaart de volgorde van de elementen.
- ☐ waar ☐ niet waar
8. In een **set** kan hetzelfde element twee keer staan.
- ☐ waar ☐ niet waar
9. Een **string** is een rij tekens.
- ☐ waar ☐ niet waar

6 Algoritmische technieken

Schrijf bij elke omschrijving de techniek.

10. Welke techniek?

- a) een functie die zichzelf oproept _____
- b) opsplitsen en de deeloplossingen samenvoegen _____
- c) tussenresultaten bewaren om ze niet twee keer te berekenen

- d) in een gesorteerde lijst telkens de helft wegnemen

7 Recursie opschrijven

Schrijf de formule op, met het stopgeval erbij.

11. De faculteit.

12. De rij van Fibonacci.

8 Reken de recursie uit

Werk met $n! = n \cdot (n - 1)!$ en $F(n) = F(n - 1) + F(n - 2)$.

13. Hoeveel?

- a) $4!$ _____ b) $6!$ _____ c) $F(5)$ _____
d) $F(8)$ _____

9 Hoeveel stappen om te zoeken?

Je zoekt in een gesorteerde rij en gooit elke stap de helft weg. Hoeveel stappen heb je hoogstens nodig?

14. Hoeveel stappen?

- a) 8 getallen _____ b) 16 getallen _____
c) 64 getallen _____ d) 1024 getallen _____
-

10 Dynamisch programmeren

Antwoord in volle zinnen.

15. Welke oproepen heeft een programma nodig om $F(4)$ recursief uit te rekenen, en welke waarden berekent het meer dan één keer?

16. Dynamisch programmeren kost meer geheugen. Wat levert het op, en wanneer is dat de moeite?

11 Correctheid en eindigheid

Antwoord in volle zinnen.

17. Leg het verschil uit tussen de correctheid en de eindigheid van een algoritme.

18. Hoe beargumenteer je de eindigheid van een lus, en hoe die van een recursieve functie?

12 Waar of niet waar: algoritmen

Kruis aan.

19. Een algoritme is pas een algoritme als het in Python geschreven staat.

☐ waar ☐ niet waar

20. Twee algoritmen die hetzelfde antwoord geven, zijn ook even snel.

☐ waar ☐ niet waar

21. Een algoritme is eindig als het na een eindig aantal stappen stopt.

☐ waar ☐ niet waar

22. Je test je programma ook met randgevallen, omdat fouten zich daar verstoppen.

☐ waar ☐ niet waar

13 Bibliotheken en bestanden

Vul aan.

23.

Bibliotheek	Waarvoor je ze gebruikt
matplotlib	_____
numpy	_____
random	_____

24. Welk soort bestand lees je naast een gewoon tekstbestand in en schrijf je weg?

14 Groeigedrag

Vul in hoeveel stappen je ongeveer doet bij $n = 1000$.

25.

Groeigedrag	Ongeveer hoeveel stappen bij $n = 1000$
$O(\log n)$	_____
$O(n)$	_____
$O(n^2)$	_____

15 De Riemannsom

Schrijf je berekening volledig uit.

26. Schrijf de benadering van $\int_a^b f(x) dx$ op als een som, en zeg wat Δx is.

27. Je verdeelt $[0, 4]$ in 8 stroken. Hoe breed is elke strook, en wat gebeurt er met de benadering als je n verdubbelt?

16 Uitleggen

Schrijf je uitleg in volle zinnen.

- 28.** Waarom geef je variabelen een zinvolle naam, en waarvoor dient commentaar in je code?

- 29.** Noem de vier stappen waarin je een oplossing uitwerkt, en leg uit waarom je niet meteen begint te typen.

17 Doordenkers

Denk eerst na, schrijf daarna je redenering op.

- 30.** Een lijst van een miljoen namen is niet gesorteerd en je moet er één naam in terugvinden. Mag je hier halveren?

- 31.** Twee programma's geven hetzelfde antwoord. Het ene doet $O(n)$ stappen, het andere $O(n^2)$. Bij $n = 10$ merk je nauwelijks verschil. Hoe komt dat, en wanneer wel?

Antwoorden

Dit blad is voor wie meekijkt. Scheur het eraf voor je de bundel geeft.

Een fout antwoord is het interessantste stuk van de bundel: vraag hoe je kind eraan kwam voor je zegt wat juist is. Vaak zit de vergissing één stap eerder dan je denkt.

1. De bouwstenen

- 1** a) een variabele · b) een constante · c) een conditie · d) een iteratie
2 een functie

2. Het juiste sleutelwoord

- 3** a) if · b) for · c) def · d) return

3. De juiste datastructuur

- 4** een list; een dictionary; een set; een tuple

4. Wat blijft er over in een set?

- 5** a) 3 · b) 4 · c) 1 · d) 3

5. Waar of niet waar: datastructuren

- 6** niet waar
7 waar
8 niet waar
9 waar

6. Algoritmische technieken

- 10** a) recursie · b) verdeel-en-heers · c) dynamisch programmeren · d) verdeel-en-heers

7. Recursie opschrijven

- 11** $n! = n \cdot (n - 1)!$, met $0! = 1$ als stopgeval.
12 $F(n) = F(n - 1) + F(n - 2)$, met $F(0) = 0$ en $F(1) = 1$.

8. Reken de recursie uit

- 13** a) 24 · b) 720 · c) 5 · d) 21

9. Hoeveel stappen om te zoeken?

- 14** a) 3 · b) 4 · c) 6 · d) 10

10. Dynamisch programmeren

- 15** $F(4)$ roept $F(3)$ en $F(2)$ op, $F(3)$ roept $F(2)$ en $F(1)$ op, en $F(2)$ roept telkens $F(1)$ en $F(0)$ op. Zo bereken je $F(2)$ twee keer en $F(1)$ drie keer.
16 Het wint tijd, want je berekent niets twee keer. Bij een probleem waar dezelfde deelberekening vaak terugkomt, zoals de rij van Fibonacci, scheelt dat enorm.

11. Correctheid en eindigheid

- 17** Correct betekent dat het voor elke toegelaten invoer het juiste antwoord geeft. Eindig betekent dat het na een eindig aantal stappen stopt. Een programma kan correct geredeneerd zijn en toch eeuwig blijven lopen.
18 Bij een lus toon je dat de voorwaarde ooit vals wordt. Bij recursie toon je dat je het stopgeval altijd bereikt.

12. Waar of niet waar: algoritmen

- 19** niet waar
20 niet waar
21 waar
22 waar

13. Bibliotheken en bestanden

- 23** grafieken tekenen; vlot met grote rijen getallen rekenen; toevalsgetallen genereren
24 een CSV-bestand

14. Groeigedrag

- 25** ongeveer 10; ongeveer 1000; ongeveer 1 000 000

15. De Riemannsom

- 26** $\int_a^b f(x) dx \approx \sum_{i=1}^n f(x_i) \cdot \Delta x$, met $\Delta x = \frac{b-a}{n}$, de breedte van één strook.
- 27** $\Delta x = \frac{4-0}{8} = 0,5$. Verdubbel je n , dan worden de stroken half zo smal en wordt de benadering beter.

16. Uitleggen

- 28** Allebei om je code leesbaar te houden voor jezelf en voor anderen, en dat telt mee in de beoordeling. Commentaar wordt niet mee uitgevoerd; ze staat er alleen voor wie de code leest.

- 29** Analyseren, een algoritme ontwerpen, programmeren, en testen en debuggen. Meteen beginnen typen zonder het probleem te analyseren kost je achteraf meer tijd dan het wint.

17. Doordenkers

- 30** Nee: halveren werkt alleen op een gesorteerde rij. Je loopt de lijst dan van voor naar achter door, of je sorteert ze eerst als je nog vaak moet zoeken.
- 31** Bij $n = 10$ is dat 10 tegenover 100 stappen, en dat duurt allebei niets. Bij $n = 1000$ wordt het 1000 tegenover 1 000 000, en dan wordt het tweede onbruikbaar.